

Industrialisation, durcissement et architecture ci/cd

Contexte du Projet

L'équipe de développement vous confie le dépôt d'une application. L'architecture est scindée en deux composants distincts au sein d'un même dépôt :

- `/frontend` : Une Single Page Application (SPA) entièrement statique (HTML, CSS, JavaScript moderne) qui consomme l'API.
- `/backend` : Une API REST développée en Node.js / Express manipulant des données hautement sensibles, nécessitant des clés d'API et des accès à des infrastructures externes.

Architecture du dépôt et gouvernance git

Avant d'écrire la moindre ligne de code ou de pipeline, votre équipe doit structurer la gouvernance du dépôt Git pour interdire toute action non audité. Vous devez implémenter un flux où le code de production est techniquement validé avant son déploiement.

1. Configuration des Branches

- **Branche `staging`** : C'est la branche pivot de développement et d'intégration. Tout commit ou Pull Request des développeurs doit y être fusionné. La CI s'y exécute entièrement à chaque événement.
- **Branche `main` (production)** : Cette branche représente l'état stable en production. **Les push directs y sont strictement interdits.**

2. Visibilité de la Protection dans le Fichier YAML

Pour garantir que la politique de sécurité de l'entreprise est transparente et immuable, **la configuration de votre pipeline doit afficher explicitement la stratégie de promotion de code**. Votre fichier de workflow devra obligatoirement :

- Être déclenché à la fois sur `staging` et sur `main` (`on: push: branches: [staging, main]`).
- Utiliser des blocs conditionnels de niveau job (`if: github.ref == 'refs/heads/main'`) combinés à une matrice de dépendance stricte (`needs`).
- Lier explicitement le job de production à un environnement GitHub nommé (`environment: production`).



Tip

À la simple lecture de votre fichier YAML, un auditeur doit constater qu'un déploiement sur `main` exige le succès absolu des jobs de validation exécutés au préalable.

Validation stricte, cryptographie, conteneurisation et ci hermétique

Environnement local

Avant qu'une seule ligne de code n'atteigne GitHub, vous devez garantir l'intégrité des commits sur le poste de travail (concept *Shift Left*).

1. **Hook pre-commit** : Créez un script `.git/hooks/pre-commit` . Il doit obligatoirement valider séquentiellement :
 - `actionlint` sur l'ensemble des fichiers du dossier `.github/workflows/` .
 - Un scan `gitleaks` local focalisé uniquement sur les modifications prêtes à être committées.
2. **Règles gitleaks sur-mesure** : Ajoutez un fichier `gitleaks.toml` à la racine. Concevez une expression régulière personnalisée pour intercepter les jetons d'accès internes de l'entreprise qui commencent par le préfixe `SECWALLET_` suivi d'exactly 24 caractères alphanumériques en majuscules, combinée à une vérification d'entropie.
3. **Contrôle de structure strict** : En plus des scanners, votre hook `pre-commit` doit inspecter la liste des fichiers modifiés. Si un fichier se terminant par `.env` , `.pem` ou `.key` est détecté dans l'index Git, le commit doit être avorté immédiatement avec un message d'erreur rouge explicite : *"Sécurité : Tentative de commit d'un fichier de configuration ou d'une clé en clair. Opération annulée."*

Gestion des secrets par enveloppe

La philosophie GitOps exige qu'aucun secret de production ne soit stocké en clair, mais l'équipe Ops doit pouvoir auditer les structures de fichiers.

1. Générez une paire de clés asymétriques via l'utilitaire `age` . La clé privée doit être nommée `ops.txt` .
2. Créez un fichier `.github/secrets-prod.yaml` contenant les clés de production de l'API actuelle et futur (ex: `DATABASE_URL` , `JWT_SECRET`).
3. Chiffrez ce fichier avec `SOPS` de sorte que seules les valeurs applicatives soient transformées en blocs `ENC[...]` , tandis que les clés structurelles YAML doivent rester lisibles pour permettre des `git diff` propres.

Conteneurisation multi-stage, filtrage et livraison sur ghcr

Vous devez automatiser la création et la publication de l'artefact serveur du backend sous forme d'image Docker sécurisée.

1. **Rédaction du Dockerfile** : À l'intérieur du dossier `backend/` , concevez un `Dockerfile` en respectant les bonnes pratiques vues en cours.
2. **Optimisation et filtrage de chemins** : Dans votre pipeline de CI, configurez l'étape de build Docker pour (re)construire l'image que si les fichiers concernés par celle-ci ont subi des modifications.
3. **Scan de sécurité de l'image** : Avant toute publication sur un registre, intégrez une étape de scan de l'image de build locale à l'aide d'un scanner de conteneurs (ex: Trivy ou Gype en mode image).
4. **Publication conditionnelle sur GHCR** : Si et seulement si le scan de l'image se termine avec succès (aucune vulnérabilité haute ou critique), configurez le pipeline pour qu'il s'authentifie dynamiquement auprès de **GHCR** (GitHub Container Registry) et y pousse l'image finale correctement tagguée avec le SHA du commit Git.

Pipeline de ci durci et interruption absolue

Créez votre workflow principal dans `.github/workflows/ci-cd.yml` .

1. **Principe du moindre privilège** : Positionnez un bloc `permissions:` global au niveau du workflow configuré à `contents: read` uniquement. Aucun job ne doit hériter de droits d'écriture par défaut. Les privilèges requis comme `packages: write` devront être isolés localement au niveau du job concerné.
2. **Optimisation avancée** : Mettez en place une action de mise en cache pour les nœuds de dépendances de Node.js.
3. **Analyse statique de sécurité** : Intégrez le moteur CodeQL pour analyser le code JavaScript. Pensez à téléverser les rapports SARIF.

4. **Politique d'interruption stricte** : Vous devez configurer votre pipeline pour qu'il agisse comme une barrière.
- **Testing** : Exécutez les tests fournis par l'équipe de développement et vérifiez qu'ils ne retournent pas d'erreur pour continuer l'exécution de la pipeline.
 - **Gitleaks en CI** : L'étape Gitleaks doit renvoyer un code de sortie non nul si un secret est découvert. L'usage de `continue-on-error: true` est interdit.
 - **CodeQL / SAST** : Configurez le job de scan de telle sorte que si une vulnérabilité de sécurité majeure (High/Error) est détectée dans le code source du backend, le job soit marqué en échec.
 - **Conséquence sur la CD** : Les jobs de déploiement dépendront directement de la validation complète de ces étapes. Si Gitleaks lève une alerte, si CodeQL trouve une faille de sécurité, ou si le scan d'image Docker échoue, le pipeline doit s'arrêter et bloquer immédiatement la suite du workflow et tout déploiement.

Création d'une "Composite Action" GitHub

Pour centraliser la gouvernance de la sécurité et éviter la redondance de code, vous allez isoler vos scanners de dépendances au format SBOM.

Qu'est-ce qu'une Action Composite GitHub ?

Une **Action Composite** vous permet de packager plusieurs étapes de workflow (`steps`) au sein d'une seule et même action réutilisable. Contrairement à un workflow classique, elle possède son propre fichier de configuration structurel (`action.yml`) et s'exécute dans le même contexte que le job qui l'appelle. Elle permet de centraliser des commandes répétitives, de masquer la complexité opérationnelle pour les développeurs, et de définir des entrées (`inputs`) et sorties (`outputs`) strictes pour interagir avec le pipeline principal.

Au lieu d'appeler Trivy directement dans votre workflow principal pour l'analyse, créez une action locale personnalisée dans votre dépôt, située dans le dossier `.github/actions/trivy-scan/action.yml`.

- Cette action doit être de type `composite`.
- Elle doit accepter un paramètre obligatoire (`inputs`) : le chemin du SBOM au format CycloneDX JSON généré au préalable dans une étape amont.
- Elle doit exécuter le binaire Trivy en mode analyse de SBOM.
Le build du pipeline doit échouer **uniquement** si des vulnérabilités de criticité `CRITICAL` sont détectées. Si des vulnérabilités `HIGH` ou `MEDIUM` sont présentes, l'action doit générer un avertissement dans le résumé du workflow mais laisser le pipeline continuer.

Déploiement Continu

Le pipeline de déploiement ne doit se déclencher **que si l'intégralité des jobs de la CI sont validés avec succès**, et uniquement lors d'un push/merge validé sur la branche `main`.

CD frontend sur github pages via artefacts éphémères

Tip

vous ne devez pas utiliser le déploiement classique basé sur une branche dédiée (comme `gh-pages`). Vous devez utiliser l'API moderne d'artefacts de déploiement de GitHub.

1. Configurez votre projet GitHub pour que la source des Pages soit définie sur "**GitHub Actions**" (Paramètres > Pages > Build and deployment > Source).
2. Dans votre fichier de workflow, créez un job `deploy-frontend` qui s'exécute uniquement sur la branche `main`.
3. Ce job doit exiger des privilèges spécifiques via OpenID Connect (OIDC). Vous devez lui attribuer explicitement les permissions `pages: write` et `id-token: write`.
4. Utilisez l'action officielle `actions/upload-pages-artifact` pour packager le contenu de votre dossier `/frontend`.
5. Utilisez ensuite l'action `actions/deploy-pages` pour propulser le site en production de manière totalement hermétique, sans laisser de traces historiques de build dans l'historique Git.

CD backend sur Vercel



Tip

Le déploiement automatisé du backend Node.js va s'effectuer sur la plateforme Vercel. Vous devez orchestrer ce déploiement directement en ligne de commande au sein de GitHub Actions.

1. **Authentification machine-to-machine** : Créez un compte sur Vercel, récupérez votre `VERCEL_TOKEN` et votre `VERCEL_PROJECT_ID` / `VERCEL_ORG_ID`. Intégrez ces valeurs dans les secrets chiffrés de votre dépôt GitHub.
2. **Job `deploy-backend`** : Ce job doit exiger le succès complet des jobs de sécurité et de livraison.
3. **Déchiffrement au runtime** :
 - Récupérez le secret GitHub contenant la clé privée Age (`SOPS_AGE_KEY`).
 - Exécutez le déchiffrement dynamique du fichier `.github/secrets-prod.yaml` à l'aide de SOPS directement en mémoire RAM.
 - Les secrets déchiffrés doivent être injectés à la volée dans la commande de déploiement Vercel pour que l'API distante dispose de ses variables de production. Aucun fichier contenant des secrets en clair ne doit être écrit sur le disque dur du runner GitHub.

Robustesse, concurrence et stratégie de rollback

Pour valider la résilience industrielle de votre infrastructure :

1. **Gestion de la concurrence** : Si un développeur pousse deux commits coup sur coup, le pipeline du premier commit doit s'annuler immédiatement pour ne pas gaspiller de ressources et éviter la concurrence lors des déploiements.
2. **Healthcheck post-déploiement** : Ajoutez une étape finale dans votre CD backend. Une fois le déploiement Vercel terminé, récupérez l'URL de production générée dynamiquement. Le pipeline doit exécuter une requête `curl` de diagnostic vers le endpoint `/api/health` de cette URL. Si l'API renvoie un code autre que `200 OK`, le job GitHub Actions doit échouer, notifiant immédiatement l'équipe d'une anomalie au déploiement.

Livrables

Chaque groupe devra déposer sur Moodle un unique fichier texte avec les URLs Vercel et GitHub Pages et l'URL vers le dépôt GitHub public contenant :

1. **L'arborescence complète et fonctionnelle** : Le frontend, le backend, les hooks locaux personnalisés, et la structure `.github/` au complet montrant clairement la protection, les conditions de filtrage Docker, et le chaînage logique des dépendances.

2. **Tout autre fichier que vous jugez utile** : Capture d'écran, scripts, etc.